AI Agent Behavior Diagrams and Descriptions (Harir Al-Rubaye 1108085)

Overview

This document provides a detailed explanation of the behavior of the AI agents (Defender and Goalie) in the Stonehenge-based soccer game. It includes diagrams, code summaries, and the necessary mathematical and physics formulas used to implement their behaviors.

PlayerController Script Overview

Behavior Diagram:

Player Input (WASD) -> [Player Controller] -> Move/Rotate Player

Description:

The PlayerController script enables the player to rotate left and right and move forward/backward based on keyboard input. The player's movement is managed using a CharacterController for smooth navigation and collision handling.

[Using Simple Move example Script from the Unity documentation]

Relevant Formulas

1. **Rotation:** θ = Input.GetAxis("Horizontal") × rotateSpeed
    - **Explanation:** The rotation angle θ is determined by the horizontal input multiplied by the rotation speed. This updates the player's orientation.
2. **Linear Velocity:** v = speed × Input.GetAxis("Vertical")
    - **Explanation:** The player's speed v in the forward direction is controlled by the vertical input.
3. **Transformation:**

    worldDirection =transform.TransformDirection(localDirection)
    - **Explanation:** Converts a local direction vector to a world-space direction vector.

DefenderAI Script Overview

Behavior Diagram:

Player Position -> [Defender AI] -> Maintain Distance / Face Player

Description:

The Defender AI maintains a specific distance from the player, positioning itself between the player and the goalie, 2m away from the player. It moves towards the

player if the distance is greater than the desired threshold and rotates to continuously face the player.

How the Script Works

1. **Player Position Tracking:**
   - The script continuously tracks the player's position using the player's Transform.
2. **Distance Maintenance:**
   - The defender calculates the Euclidean distance to the player on the horizontal plane. The formula for distance calculation is:
     - **Distance Calculation:**

$$d = \sqrt{(x_2 - x_1)^2 + (z_2 - z_1)^2}$$

Line of Code: *float distance = directionToPlayer.magnitude;*

Explanation: Calculates the Euclidean distance d between the player and the defender on the horizontal plane.

3. **Movement Towards Player:**
   - If the distance between the defender and the player exceeds the desired threshold, the defender moves towards the player. The movement direction is

calculated by normalizing the direction vector to the player and then applying a velocity vector proportional to the distance that needs to be closed. The formulas are:

- **Normalized Direction Vector:** $\hat{v} = \frac{v}{|v|}$

  Line of Code: *Vector3 moveDirection = directionToPlayer.normalized;*

  Explanation: The direction vector v is normalized to have a magnitude of 1, preserving direction but making it unit length.

- **Velocity Vector for Movement:** $v = \hat{v} \times (\text{distance} - \text{distanceToMaintain})$

  Line of Code:

  - Vector3 moveDirection = directionToPlayer.normalized * (distance - distanceToMaintain);
  - controller.Move(moveDirection * speed * Time.deltaTime);

  Explanation: The defender moves towards the player by applying a velocity vector proportional to the distance that needs to be closed.

4. Player Facing Rotation:

- The defender continuously rotates to face the player. This involves calculating the angle required to face the player using the arctangent

function, converting this angle to a quaternion for smooth rotational interpolation, and then applying linear interpolation to achieve smooth rotation.

- **Angular Rotation Formula:**

$$\theta = \text{atan2}(z, x) \times \frac{180}{\pi}$$

Line of Code: *float angle = Mathf.Atan2(directionToPlayer.z, directionToPlayer.x) * Mathf.Rad2Deg;*

Explanation: Calculates the angle θ required for the defender to face the player based on their relative positions.

- **Quaternion Conversion Formula:**

$$\text{Quaternion.Euler}(0, \theta, 0)$$

  - **Line of Code:** *Quaternion targetRotation = Quaternion.Euler(0, -angle + 90, 0);*

Explanation: Converts Euler angles to a quaternion for smooth rotational interpolation.

- **Linear Interpolation Formula:**

$$\text{rotation} = \text{Lerp}(\text{currentRotation}, \text{targetRotation}, \text{rotationSpeed} \times \Delta t)$$

Line of Code: *transform.rotation = Quaternion.Lerp(transform.rotation, targetRotation, speed * Time.deltaTime);*

Explanation: Smoothly interpolates the goalie's rotation from its current orientation to the target orientation.

GoalieAI Script Overview

Behavior Diagram:

Player Position -> [Goalie AI] -> Face Player

Description:

The Goalie AI stays in front of the goal and rotates to face the player, adjusting its orientation smoothly to align with the player's position.

How the Script Works

1. **Player Position Tracking:** The script continuously tracks the player's position using the player's Transform.
2. **Facing the Player:** The goalie continuously rotates to face the player. The rotation angle is calculated using the arctangent function. The formula for angular rotation is:
    - **Angular Rotation Formula:** $$\theta = \text{atan2}(z, x) \times \frac{180}{\pi}$$

Line of Code: *float angle = Mathf.Atan2(directionToPlayer.z, directionToPlayer.x) * Mathf.Rad2Deg;*

Explanation: Computes the angle θ for the goalie to face the player based on their relative positions.

4. **Smooth Rotation:** The goalie's rotation is interpolated smoothly from its current orientation to the target orientation using the *Quaternion.Lerp* function. The formula for linear interpolation is:
    - **Linear Interpolation Formula:**

$$\text{rotation} = \text{Lerp}(\text{currentRotation}, \text{targetRotation}, \text{rotationSpeed} \times \Delta t)$$

Line of Code: *transform.rotation = Quaternion.Lerp(transform.rotation, targetRotation, rotationSpeed * Time.deltaTime);*

Explanation: Smoothly interpolates the goalie's rotation from its current orientation to the target orientation.

4. **Quaternion Conversion:** The calculated angle is converted into a quaternion for smooth rotational interpolation using the *Quaternion. Euler* function.
    - **Quaternion Conversion Formula:** $\text{Quaternion.Euler}(0, \theta, 0)$

Line of Code: *Quaternion targetRotation = Quaternion.Euler(0, -angle + 90, 0);*

Explanation: Converts Euler angles to a quaternion for smooth rotational interpolation.

Camera Setup and Defender Visibility

To ensure the player has a clear view of the game, I implemented a Cinemachine camera with an offset of Z = -90. The primary reason for this offset is to improve visibility of the defender. Since the defender is positioned 2m away from the player, the default camera positioning might cause the player and defender to appear too close together in the view, making it difficult for the player to clearly see the defender's position and movements.

The offset of Z = -90 positions the camera slightly behind the player, which ensures that the defender remains clearly visible while still allowing the player to navigate the scene. However, this camera setup may cause some visual discrepancies when rotating the camera, especially when the player moves. Although the defender is correctly facing the player based on the AI logic, the 360-degree camera rotation

might give the impression that the defender isn't aligned properly. This is purely a visual issue and does not affect the AI's behavior or positioning, which can be confirmed in the scene screen.

How to Run the Project

1. **Unpack the Project**
    - Download the compressed project file (.zip or .tar.gz).
    - Extract the files to a folder of your choice. The extracted folder will contain the necessary Unity project files.
2. **Open the Project in Unity**
    - Open **Unity Hub**.
    - Click on **"Open"** or **"Add Project"** and navigate to the folder where the project was extracted.
    - Select the project folder.
    - Click **Open** to load the project in Unity.
3. **Set Unity Version**

- Make sure you are using the correct version of Unity for this project (e.g., **I used 6000.0.32f1 according to the professor's instructions)**
- If you don't have the required version installed, download it through Unity Hub.

4. **Run the Project**
   - Once the project is loaded in Unity, navigate to the **Scenes** folder within the **Assets** folder.
   - Open the main scene file (the scene is called "Stonehenge").
   - Click the **Play** button in the Unity editor to run the project. This will start the scene and you can begin interacting with the project.
5. **Additional Notes**
   - **Library**, **Temp**, and **Logs** folders are excluded from the project download. These folders are temporary and will be automatically regenerated by Unity when the project is opened.
   - Ensure that your system meets Unity's minimum requirements to avoid performance issues.

Conclusion

This document outlines the AI behaviors for the Player, Defender, and Goalie agents in the Ancient Soccer Madness game. Each agent's behavior is backed by detailed code explanations and mathematical formulas, incorporating physics principles such as velocity, distance calculation, angular rotation, and smooth interpolation to ensure realistic interactions